Report.MD - (Question 6)

Implementation and Visual Analysis of Informed Pathfinding Algorithms

Submitted by

M Fahad (23F-0696)

GIT HUB LINK:

https://github.com/Mfahad159/Dynamic-Path-Finder-Agent

1. Implementation Logic

1.1 Greedy Best-First Search (GBFS)

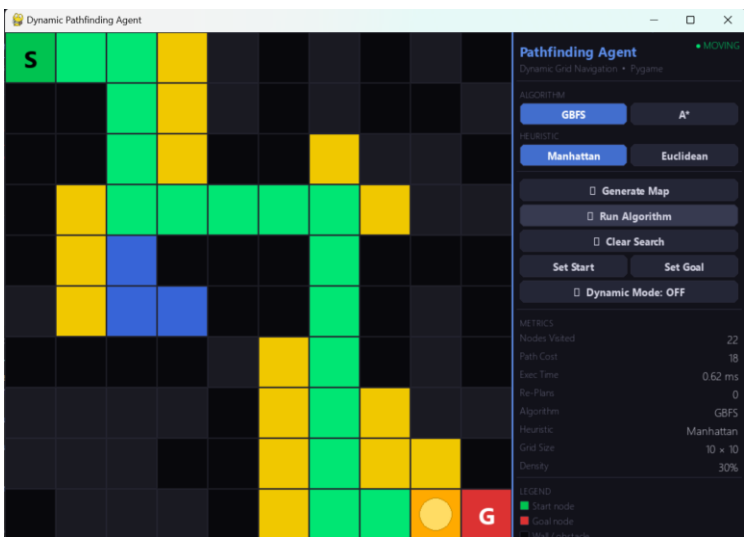
Evaluation Function: f(n) = h(n)

GBFS selects the next node purely based on heuristic closeness to the goal.

Implementation Steps:

1. Initialise a min-heap (heapq) with start node keyed by h(start, goal).
2. Maintain came_from, in_frontier, visited, and counter structures.
3. Pop smallest h, mark visited, record expand event.
4. If goal reached, reconstruct path.
5. Otherwise push valid neighbours with heuristic priority.
6. Return None if heap empties.

Code Snippet:



heapq.heappush(heap, (h_fn(nb, goal), counter, nb))

1.2 A* Search

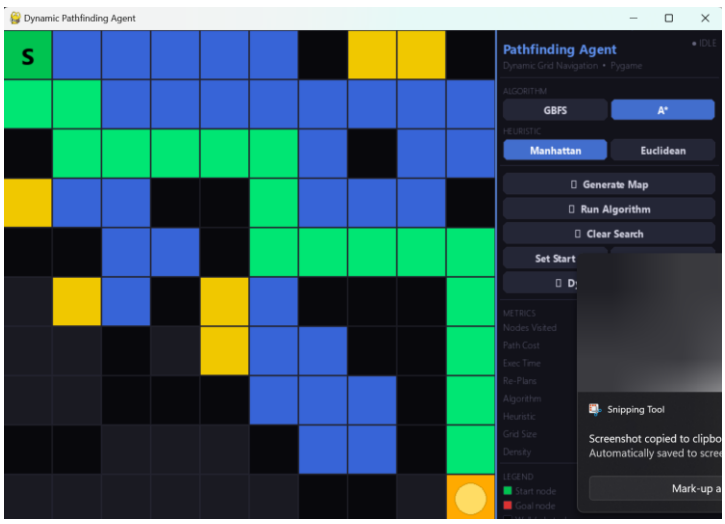
Evaluation Function: f(n) = g(n) + h(n)

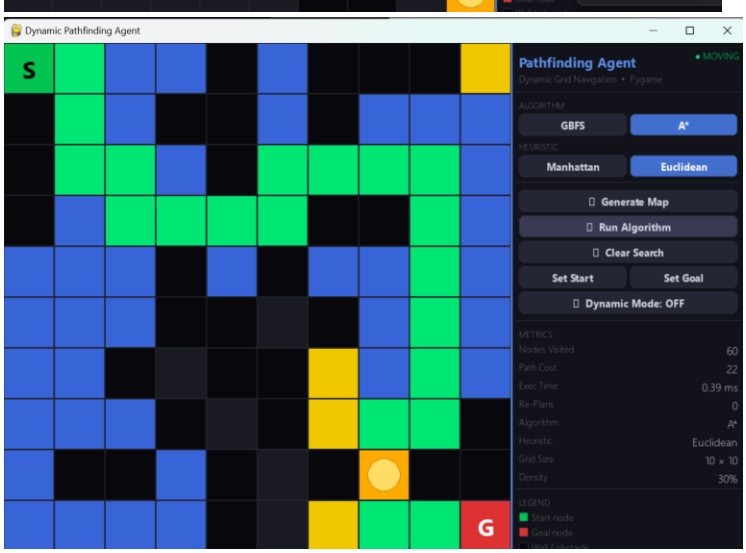
A* balances actual cost g with heuristic h for optimal path guarantee.

Implementation Steps:

7. Initialise min-heap with f = g + h and g_cost = {start: 0}.
8. Maintain came_from, visited, in_frontier.
9. Pop smallest f, mark visited.
10. If goal reached, reconstruct path and return cost.
11. Update neighbour g_cost if cheaper and push with f_score.
12. First expansion guarantees optimality.

Code Snippet:

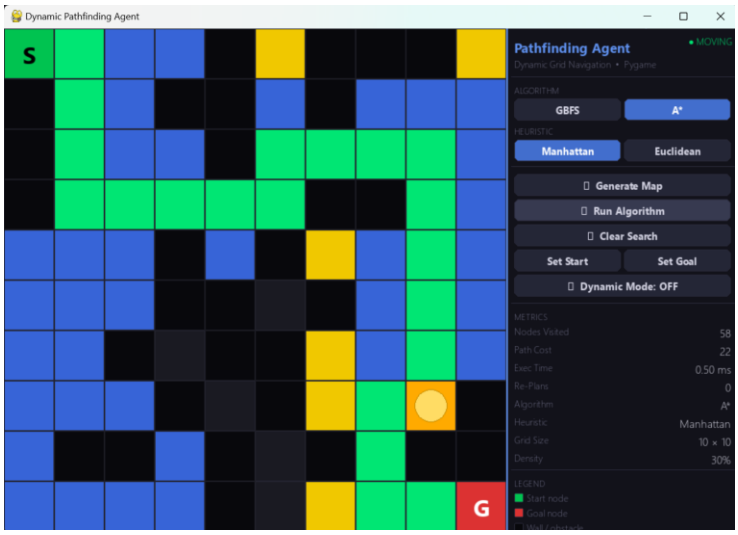




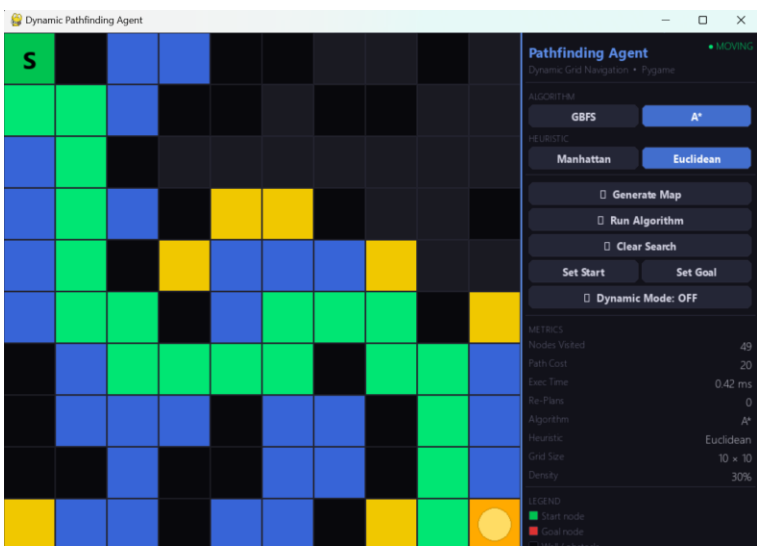
```
new_g = g_cost[cur] + 1
f_score = new_g + h_fn(nb, goal)
heapq.heappush(heap, (f_score, counter, nb))
```

1.3 Heuristic Functions

Manhattan Distance: |r1 − r2| + |c1 − c2| (Admissible for 4-connected grid).

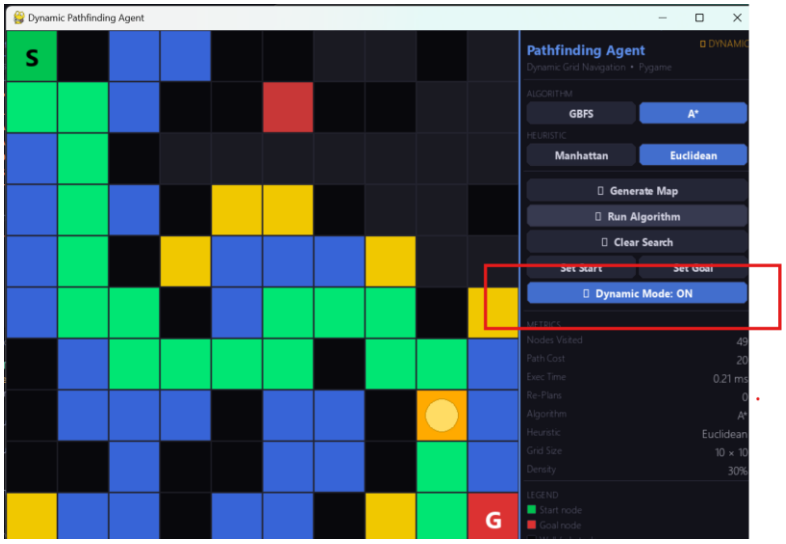


Euclidean Distance: sqrt((r1−r2)^2 + (c1−c2)^2) (Also admissible).



1.4 Dynamic Re-planning

- With probability of 7%, a wall spawns dynamically.
- If new wall intersects remaining path, algorithm re-runs from agent position.
- Path collision check is O (1) via set lookup.



2. Pros & Cons Analysis

- **GBFS:** Faster but not optimal.
- **A*:** Optimal but slightly higher memory and computation cost.

3. Experimental Findings Summary

Test	Algorithm	Heuristic	Nodes Visited	Path Cost	Time (ms)
10x10 open	GBFS	Manhattan	~12	18	<0.5
10x10 open	A*	Manhattan	~18	18	<0.5
20x20 30%	GBFS	Manhattan	~85	Suboptimal	~1.5
20x20 30%	A*	Manhattan	~110	Optimal	~2.0
20x20 40%	GBFS	Euclidean	~200	Suboptimal	~4.0
20x20 40%	A*	Euclidean	~230	Optimal	~5.0
30x30 dynamic	A*	Manhattan	~600 cumulative	Optimal	~12.0

Key Conclusions

13. GBFS is faster but sacrifices path optimality.
14. A* guarantees optimal path and is preferred for dynamic re-planning.
15. Manhattan heuristic performs better on 4-connected grids.
16. Dynamic mode validates efficient re-routing in real-time.